

Михаил Разаков

Инженер по компиляторам и средам исполнения

Разрабатываю UniversalToolchain/Wist2 — модульную .NET-платформу для компиляции и исполнения DSL с Bytecode/AIR, эталонным интерпретатором и генерацией CIL/DynamicMethod. Реализовал экспериментальный проверенный маршрут AIR → SSA → AIR: преобразования между IR, свёртку констант, SCCP-like, упрощение ветвлений, удаление недостижимого кода и мёртвых чистых инструкций. PS-form Analyzer подтверждает опыт консервативного анализа программ и проверки через точный эталон.

- Москва / удалённо
- Студент НИУ ВШЭ
- С сентября 2026: до 20 часов в неделю
- Стажировка МЦСТ - июль-август 2026

misha13022008@gmail.com · t.me/Mcody · github.com/Misha1302 · LinkedIn

AIR → SSA → AIR lowering/emission и оптимизации под контролем структурных верификаторов	1 325 тестов · 75 .NET-проектов baseline от 14.07.2026; 0 предупреждений и 0 ошибок	1-е место · 104/104 консервативный анализ зависимостей памяти
---	---	---

2024 — сейчас создатель UniversalToolchain/Wist2	2 backend-пути reference interpreter и CIL/DynamicMethod	IR → optimization → runtime явные границы и safety contracts	x86 / x86-64 Assembly NASM/FASM · ABI · x87 · compiler output
--	--	--	---

01 · ОПЫТ	Компиляторы, анализ программ и проверка семантики.
------------------	---

2024 — СЕЙЧАС	UniversalToolchain / Wist2 — создатель и основной разработчик Compiler/runtime framework · .NET - публичная alpha-версия	- Спроектировал конвейер Source → AST → Bytecode → AIR → Backend → Execution, эталонный интерпретатор, backend на CIL/DynamicMethod и модульную композицию. - Реализовал экспериментальный проверочный маршрут AIR → SSA → AIR: lowering/emission, constant folding, SCCP-like, branch folding, удаление недостижимого кода и DCE. - SSA проверяется после lowering и каждого прохода, сгенерированный AIR → период backend; независимость Interpreter/CIL проверяется для поддерживаемой семантики. - Проверенный baseline от 14.07.2026: 75 .NET-проектов собирается, 1 325 тестов проходят, 0 предупреждений и 0 ошибок.
----------------------	--	---

ИЮЛЬ-АВГУСТ 2026	МЦСТ — стажёр по разработке компиляторов LLVM-направление · C++23 - удалённо	- Разрабатываю компоненты анализа графов над общей моделью данных и явные контракты входов и ошибок. - Поддерживаю CMake, warnings-as-errors, ASan/UBSan, clang-tidy, явные include-зависимости и воспроизводимые IO-тесты.
-------------------------	--	---

2021 — СЕЙЧАС	Преподаватель программирования и наставник C/C++ · C# · Python · NASM - алгоритмы	Обучил около 50 учеников; провожу диагностику знаний, объясняю модели выполнения и ревьюю код.
----------------------	---	--

02 · ПРОЕКТЫ	Compiler/runtime, program analysis и low-level backend experiments.
---------------------	--

КОМПИЛЯТОРНАЯ ПЛАТФОРМА · .NET

UniversalToolchain / Wist2

Публичная модульная платформа для встраиваемых DSL с несколькими уровнями IR и двумя реализациями исполнения.

GitHub ↗ · Документация ↗

Конвейер
Лексер, парсер, AST, Bytecode, AIR, эталонный интерпретатор и генерация CIL/DynamicMethod.

Маршрут SSA
Неизменная модель, явные привязки вызовов, проходы с проверкой возможностей backend и контролируемый возврат к исходному маршруту.

Проверка
Структурные проверки до и после преобразований, сравнение путей исполнения и целевые регрессионные тесты.

АНАЛИЗ ПРОГРАММ · C17

PS-form Memory Dependence Analyzer

Консервативный анализатор зависимостей памяти между независимыми итерациями цикла.

- 1-е место из 49; единственные 5,0/5,0 и 104/104.
- Нормализация, range/residue/GCD-фильтры, точный аффинный анализ и ограниченный поиск.
- Точный эталон, рандомизированные и метаморфные тесты, сохранение контрипримеров.

GitHub ↗

C17 · Program analysis · 104/104

ASSEMBLY · BACKEND · NASM / RUST · IA-32 / X86-64

x86 Assembly, Codegen & Register Allocation

Ручной NASM-код для IA-32 и backend-лаборатория генерации x86-64: от ABI и стека до liveness, register allocation и проверки нативного кода.

- NASM IA-32: cdecl, стековые кадры, вызовы libc, структуры и адресация, x87; собираемые примеры и практический учебный курс.
- x86-64: SSA-подобная IR, live intervals, граф интерференции, linear scan и lowering/emission через lced-x86.
- Дизассемблирование и анализ compiler output; differential testing, метрики spills/code size и изолированный запуск.

Codegen lab ↗ · NASM course ↗

NASM/FASM · IA-32 / x86-64 · ABI / x87

БИБЛИОТЕКА АЛГОРИТМОВ · C++23

AdvancedAlgorithms

Переиспользуемая C++23-библиотека алгоритмов и структур данных с явными контрактами.

- Графы, деревья, строки и структуры данных: центроидная декомпозиция, HLD, LCA, Dinic, SCC, Aho-Corssick и другие.
- Дифференциальные тесты, структурные инварианты, негативные сценарии и ASan/UBSan.
- Проверки на крупных входах выявляют переполнение стека и случайную квадратичную сложность.

GitHub ↗

C++23 · Algorithms · Testing

03 · КОМПЕТЕНЦИЯ	Компиляторные конвейеры, анализ программ и проверка семантики.
-------------------------	---

Архитектура и IR	Лексический и синтаксический анализ, AST, Bytecode и AIR; эталонный интерпретатор и генерация CIL/DynamicMethod; модульная композиция и явные контракты возможностей.
Анализ и оптимизации	CFG и SSA; преобразования AIR → SSA и SSA → AIR; SCCP, свёртка констант и ветвлений, удаление недостижимых блоков и мёртвых чистых инструкций; анализ живости и зависимостей памяти.
Проверка корректности	Структурная проверка IR до и после преобразований, сравнение результатов интерпретатора и сгенерированного кода, точные эталоны, дифференциальные и метаморфные тесты, негативные сценарии.
Ассемблер и ABI	NASM/FASM, IA-32 и x86-64; cdecl и System V AMD64 ABI; стековые кадры, вызовы libc, структуры и адресация, x87; GDB/objdump и анализ дизассемблирования.
Языки и инструменты	C#/ .NET, C++23, C17 и Python; CMake, GitHub Actions, ASan/UBSan и clang-tidy.

04 · ДОСТИЖЕНИЯ	Олимпиадные и проектные достижения.
------------------------	--

2025-2026	Олимпиадное программирование Призёр «Высшей пробы» по олимпиадному и промышленному программированию; призёр регионального этапа ВсОШ по информатике.	«Высшая проба» ↗
2025-2026	Проекты по компиляторам Два диплома I степени конкурса «Юниор» НИЯР МИФИ и диплом I степени Балтийского научно-инженерного конкурса за compiler/runtime-проекты.	«Юниор» 2025 ↗ «Юниор» 2026 ↗ Балтийский конкурс ↗
2025-2026	«Высший пилотаж» · Computer Science Победитель направления Computer Science.	Результаты ↗

05 · ОБРАЗОВАНИЕ	Образование и доступность. Студент НИУ ВШЭ.
-------------------------	---

СТУДЕНТ	НИУ ВШЭ Москва	Студент НИУ ВШЭ.
С СЕНТЯБРЯ 2026	Стажировка или неполная занятость Москва / удалённо	Ищу задачи в compiler/runtime, program analysis, compiler testing или language tooling.

Михаил Разаков · инженер по компиляторам и средам исполнения

Обновлено 24 июля 2026